

Лабораторные работы по дисциплине

«JavaScript и веб-разработка»

Курс из 16 занятий (1 раз в неделю). Каждая лабораторная сдаётся коммитом в репозиторий `web-course-2026`, в папку `taskN/` (номер соответствует лабораторной, если не указано иное). В `README.md` каждой папки — 2–3 строки с описанием, что сделано и как запустить.

Лабораторная работа №0. Организация репозитория

Цель: подготовить единое пространство, куда будут сдаваться все последующие работы.

Что нужно сделать:

1. Зарегистрироваться на GitHub (если аккаунта ещё нет).
2. Создать **публичный** репозиторий с именем `web-course-2026`.
3. Склонировать репозиторий локально (`git clone ...`).
4. В корне создать файл `README.md`, в котором указать:
 - ФИО;
 - номер группы;
 - список папок `task0 ... task16` (создать пустыми, с одним файлом `.gitkeep` в каждой, чтобы Git отслеживал пустые папки).
5. Добавить файл `.gitignore` в корень со строками:

```
node_modules/  
.DS_Store  
dist/  
.env
```

6. Сделать коммит с сообщением `init: repository structure` и запустить в `main`.
7. Включить GitHub Pages: Settings → Pages → Source: `main / root` (или `/docs`, если так удобнее). В `task0/` положить простой `index.html` с текстом «Курс JS — <ФИО>» и убедиться, что страница открывается по ссылке `https://<username>.github.io/web-course-2026/task0/`.

Ограничения: репозиторий должен быть публичным — иначе преподаватель не сможет проверить работу.

Что сдавать: ссылку на репозиторий и ссылку на опубликованную страницу GitHub Pages (одним сообщением/в ЛМС).

Критерии проверки: репозиторий существует, README заполнен, Pages работает, история коммитов не одним гигантским коммитом «всё сразу».

Лабораторная работа №1. Семантическая вёрстка.

Страница-резюме

Цель: научиться размечать страницу семантическими тегами HTML5 и располагать блоки с помощью Flexbox без использования float и абсолютного позиционирования.

Что нужно сделать: Сверстать одностраничное резюме/портфолио (можно про себя, можно про вымышленного персонажа). Страница обязана содержать следующие блоки:

1. **Шапка** (`<header>`) — имя, должность/специальность, короткий слоган.
2. **Фотография/аватар** — любое изображение (можно placeholder, например через `https://i.pravatar.cc/300`).
3. **Блок «О себе»** (`<section>` или `<article>`) — 3–5 предложений биографии.
4. **Блок «Образование»** — список (`<ul>` / `<ol>`) с местом и годами обучения.
5. **Блок «Навыки»** — список в виде тегов/чипсов (например, каждый навык — `<span class="skill">`, оформленный как «пилюля» с фоном и скруглением).
6. **Блок «Контакты»** (`<footer>` или отдельная `<section>`) — минимум 3 интерактивные ссылки: `mailto:`, ссылка на соцсети/GitHub, ссылка на телефон (`tel:`). Ссылки должны иметь ховер-эффект (смена цвета/подчёркивания через `:hover`).
7. **Навигация** (`<nav>`) — если страница длинная, добавить якорные ссылки на блоки (`<a href="#skills">`).

Технические требования (обязательные):

- Ровно один файл `index.html` и один файл `style.css` в папке `task1/`. Никакого JavaScript.
- HTML5-доктайп, `<meta charset="utf-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1">`.
- Использовать **только семантические теги** для структуры: `<header>`, `<main>`, `<section>`, `<article>`, `<nav>`, `<footer>`. Использование «голого» `<div>` для структурных блоков — минус к оценке (внутри блоков `<div>` для вспомогательной вёрстки — можно).
- Вся раскладка блоков (расположение секций друг относительно друга, выравнивание элементов внутри блока) — **только через `display: flex`**. Использование `float` или `position: absolute/fixed` для основной раскладки — запрещено и обнуляет часть оценки за это задание. (Точечное использование `position: relative` для мелких декоративных элементов допустимо.)
- Обязательно продемонстрировать в CSS: `justify-content`, `align-items`, `flex-direction`, `flex-wrap`, `gap`.
- `box-sizing: border-box` должен быть выставлен глобально (`* { box-sizing: border-box; }`).
- Стили — во внешнем файле `style.css`, инлайн-стили (`style="..."`) и `<style>` внутри HTML — не использовать (кроме исключительных случаев, которые нужно уметь объяснить).

Что НЕ надо делать: не нужна адаптивность под мобильные устройства и медиазапросы — это следующая лабораторная. Не нужен JavaScript.

Что сдавать: папку `task1/` с `index.html`, `style.css`, изображениями (если локальные). Работа должна открываться через GitHub Pages по ссылке `.../task1/`.

Критерии проверки: валидная семантическая структура, отсутствие `float/absolute` в раскладке, использование Flexbox с как минимум 4 разными свойствами, отсутствие инлайн-стилей, страница открывается без ошибок в консоли.

Лабораторная работа №2. Адаптивная вёрстка. Лендинг по макету

Цель: освоить CSS Grid, подход Mobile First и медиазапросы для адаптации под три класса устройств.

Материалы: макет в Figma выдаётся преподавателем (одностраничный лендинг — услуга/продукт/мероприятие на выбор).

Что нужно сделать:

1. Сверстать лендинг с адаптацией под три контрольные точки:
 - мобильный: до 767px;
 - планшет: 768px – 1199px;
 - десктоп: от 1200px.
2. Основная структурная сетка страницы (расположение крупных блоков: шапка, герой-секция, карточки услуг/товаров, футер и т.д.) — через **CSS Grid** (`display: grid`, `grid-template-columns`, `grid-template-rows`, `gap`, `grid-column`/`grid-row`).
3. Раскладка внутри отдельных компонентов (например, содержимое одной карточки, навигационное меню, кнопки с иконкой) — через **Flexbox**.
4. Меню должно визуально превращаться в «гамбургер» на мобильной ширине (иконку можно нарисовать через `<span>` + CSS или SVG). **JavaScript-логику клика реализовывать не нужно** — только то, как это должно выглядеть (пункт 4 из требований проверяется по CSS, не по функциональности).
5. Добавить форму обратной связи: минимум поля «Имя», «Email», «Сообщение» и кнопка «Отправить». Оформить `input`, `textarea`, `button` со стилями для обычного состояния, `:hover`, `:focus`. Отправка формы не реализуется (нет действия `action`, нет JS) — только вёрстка.
6. Добавить хотя бы один плавный переход (`transition`) — например, изменение цвета кнопки при наведении, или `transform: scale()` для карточки при `hover`.

Технические требования (обязательные):